
Joins (SQL & Core)

Joining data is an important part of many of our pipelines, and both Spark core and SQL support the same fundamental types of joins. While joins are very common and powerful, they warrant special performance consideration as they may require large network transfers or even create data sets beyond our capability to handle.⁵ In core Spark it can be more important to think about the ordering of operations, since the DAG optimizer, unlike the SQL optimizer isn't able to re-order or push down filters.

Core Spark Joins

In this section we will go over the RDD type joins. Joins in general are expensive since they require that corresponding keys from each RDD are located at the same partition so that they can be combined locally. If the RDDs do not have known partitioners, they will need to be shuffled so that both RDDs share a partitioner and data with the same keys lives in the same partitions, as shown in [Figure 4-1](#). If they have the same partitioner, the data may be colocated, as in [Figure 4-3](#), so as to avoid network transfer. Regardless of if the partitioners are the same, if one (or both) of the RDDs have a known partitioner only a narrow dependency is created, as in [Figure 4-2](#). As with most key-value operations, the cost of the join increases with the number of keys and the distance the records have to travel in order to get to their correct partition.

⁵ As the saying goes, the cross product of big data and big data is an out of memory exception.

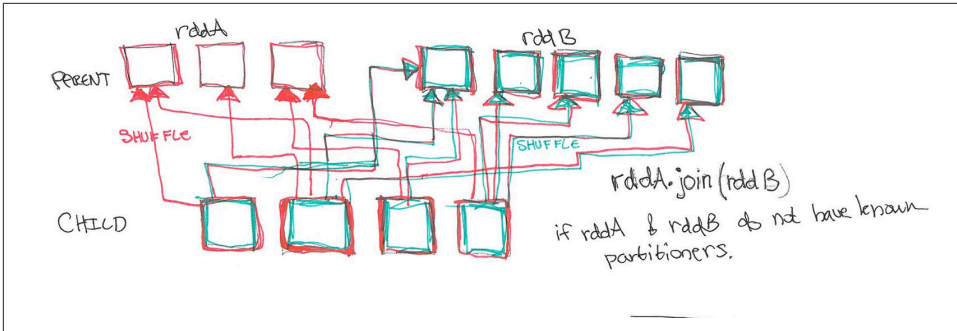


Figure 4-1. Shuffle join

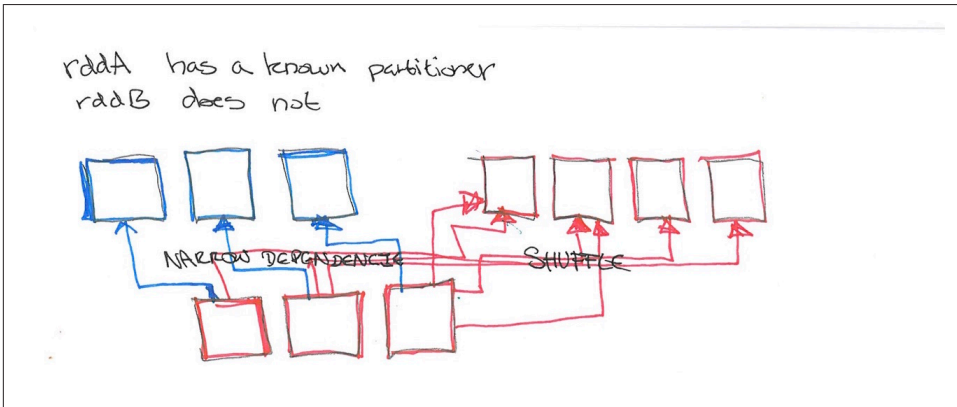


Figure 4-2. Both known partitioner join

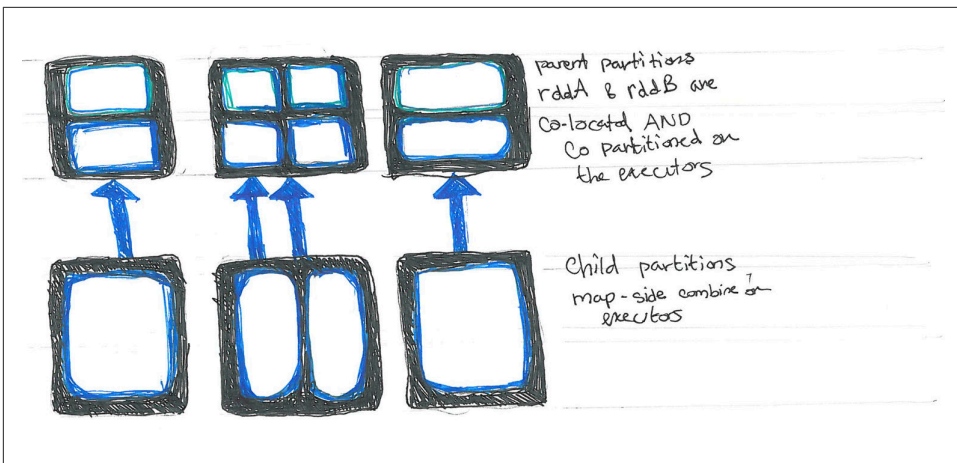


Figure 4-3. Colocated join



Two RDDs will be colocated if they have the same partitioner and were shuffled as part of the same action.



Core Spark joins are implemented using the `coGroup` function. We discuss `coGroup` in ???.

Choosing a Join Type

The default join operation in Spark includes only values for keys present in both RDDs, and in the case of multiple values per key, provides all permutations of the key/value pair. The best scenario for a standard join is when both RDDs contain the same set of distinct keys. With duplicate keys, the size of the data may expand dramatically causing performance issues, and if one key is not present in both RDDs you will lose that row of data. Here are a few guidelines:

1. When both RDDs have duplicate keys, the join can cause the size of the data to expand dramatically. It may be better to perform a `distinct` or `combineByKey` operation to reduce the key space or to use `cogroup` to handle duplicate keys instead of producing the full cross product. By using smart partitioning during the combine step, it is possible to prevent a second shuffle in the join (we will discuss this in detail later).
2. If keys are not present in both RDDs you risk losing your data unexpectedly. It can be safer to use an outer join, so that you are guaranteed to keep all the data in either the left or the right RDD, then filter the data after the join.
3. If one RDD has some easy-to-define subset of the keys, in the other you may be better off filtering or reducing before the join to avoid a big shuffle of data, which you will ultimately throw away anyway.



Join is one of the most expensive operations you will commonly use in Spark, so it is worth doing what you can to shrink your data before performing a join.

For example, suppose you have one RDD with some data in the form (Panda id, score) and another RDD with (Panda id, address), and you want to send each Panda some mail with her best score. You could join the RDDs on id and then compute the best score for each address. Like this:

Example 4-1. Basic RDD join

```
def joinScoresWithAddress1( scoreRDD : RDD[(Long, Double)],
  addressRDD : RDD[(Long, String )]) : RDD[(Long, (Double, String))]= {
  val joinedRDD = scoreRDD.join(addressRDD)
  joinedRDD.reduceByKey( (x, y) => if(x._1 > y._1) x else y )
}
```

However, this is probably not as fast as first reducing the score data, so that the first dataset contains only one row for each Panda with her best score, and then joining that data with the address data.

Example 4-2. Pre-filter before join

```
def joinScoresWithAddress2( scoreRDD : RDD[(Long, Double)],
  addressRDD : RDD[(Long, String )]) : RDD[(Long, (Double, String))]= {
  //stuff
  val bestScoreData = scoreRDD.reduceByKey((x, y) => if(x > y) x else y)
  bestScoreData.join(addressRDD)
}
```

If each Panda had 1000 different scores then the size of the shuffle we did in the first approach was 1000 times the size of the shuffle we did with this approach!

If we wanted to we could also perform a left outer join to keep all keys for processing even those missing in the right RDD by using `leftOuterJoin` in place of `join`. Spark also has `fullOuterJoin` and `rightOuterJoin` depending on which records we wish to keep. Any missing values are `None` and present values are `Some('x')`.

Example 4-3. Basic RDD left outer join

```
def outerJoinScoresWithAddress( scoreRDD : RDD[(Long, Double)],
  addressRDD : RDD[(Long, String )]) : RDD[(Long, (Double, Option[String]))]= {
  val joinedRDD = scoreRDD.leftOuterJoin(addressRDD)
  joinedRDD.reduceByKey( (x, y) => if(x._1 > y._1) x else y )
}
```

Choosing an Execution Plan

In order to join data, Spark needs the data that is to be joined (i.e. the data based on each key) to live on the same partition. The default implementation of join in Spark is a *shuffled hash join*. The shuffled hash join ensures that data on each partition will contain the same keys by partitioning the second dataset with the same default partitioner as the first, so that the keys with the same hash value from both datasets are in the same partition. While this approach always works, it can be more expensive than necessary because it requires a shuffle. The shuffle can be avoided if:

1. Both RDDs have a known partitioner.
2. One of the datasets is small enough to fit in memory, in which case we can do a broadcast hash join (we will explain what this is later).

Note that if the RDDs are colocated the network transfer can be avoided, along with the shuffle.

Speeding Up Joins by Assigning a Known Partitioner

If you have to do an operation before the join that requires a shuffle, such as `aggregateByKey` or `reduceByKey`, you can prevent the shuffle by adding a hash partitioner with the same number of partitions as an explicit argument to the first operation and persisting the RDD before the join. You could make the example in the previous section even faster, by using the partitioner for the address data as an argument for the `reduceByKey` step.

Example 4-4. Known partitioner join

```
def joinScoresWithAddress3( scoreRDD : RDD[(Long, Double)],
  addressRDD : RDD[(Long, String)] ) : RDD[(Long, (Double, String))]= {
  //if addressRDD has a known partitioner we should use that,
  //otherwise it has a default hash partitioner, which we can reconstruct by getting the number of
  // partitions.
  val addressDataPartitioner = addressRDD.partitioner match {
    case (Some(p)) => p
    case (None) => new HashPartitioner(addressRDD.partitions.length)
  }
  val bestScoreData = scoreRDD.reduceByKey(addressDataPartitioner, (x, y) => if(x > y) x else y)
  bestScoreData.join(addressRDD)
}
```

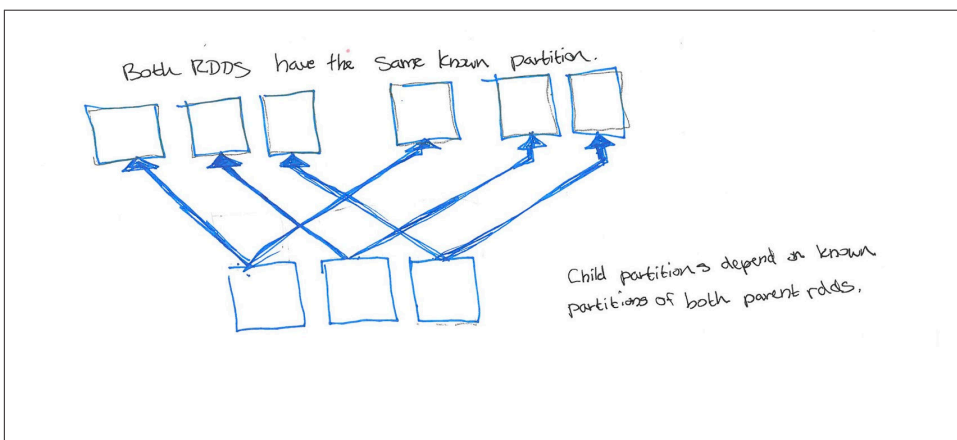


Figure 4-4. Both known partitioner join



Always persist after re-partitioning.

Speeding Up Joins Using a Broadcast Hash Join

A broadcast hash join pushes one of the RDDs (the smaller one) to each of the worker nodes. Then it does a map-side combine with each partition of the larger RDD. If one of your RDDs can fit in memory or can be made to fit in memory it is always beneficial to do a broadcast hash join, since it doesn't require a shuffle. Sometimes (but not always) Spark will be smart enough to configure the broadcast join itself. You can see what kind of join Spark is doing using the `toDebugString()` function.

Example 4-5. debugString

```
scoreRDD.join(addressRDD).toDebugString
```

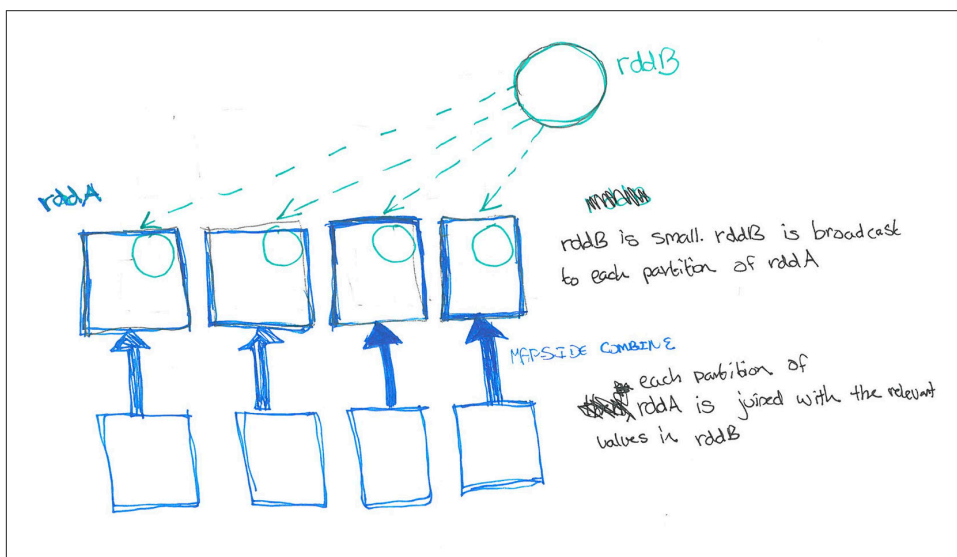


Figure 4-5. Broadcast Hash Join

Partial Manual Broadcast Hash Join

Sometimes not all of our smaller RDD will fit into memory, but some keys are so over-represented in the large data set, so you want to broadcast just the most common keys. This is especially useful if one key is so large that it can't fit on a single

partition. In this case you can use `countByKeyApprox`⁵ on the large RDD to get an approximate idea of which keys would most benefit from a broadcast. You then filter the smaller RDD for only these keys, collecting the result locally in a `HashMap`. Using `sc.broadcast` you can broadcast the `HashMap` so that each worker only has one copy and manually perform the join against the `HashMap`. Using the same `HashMap` you can then filter our large RDD down to not include the large number of duplicate keys and perform our standard join, unioning it with the result of our manual join. This approach is quite convoluted but may allow you to handle highly skewed data you couldn't otherwise process.

Spark SQL Joins

Spark SQL supports the same basic join types as core Spark, but the optimizer is able to do more of the heavy lifting for you - although you also give up some of our control. For example, Spark SQL can sometimes push down or re-order operations to make our joins more efficient. On the other hand, you don't control the partitioner for `DataFrames` or `Datasets`, so you can't manually avoid shuffles as you did with core Spark joins.

DataFrame Joins

Joining data between `DataFrames` is one of the most common multi-`DataFrame` transformations. The standard SQL join types are supported and can be specified as the "joinType" when performing a join. As with joins between RDDs, joining with non-unique keys will result in the cross product (so if the left table has R1 and R2 with key1 and the right table has R3 and R5 with key1 you will get (R1, R3), (R1, R5), (R2, R3), (R2, R5)) in the output. While we explore Spark SQL joins we will use two example tables of pandas, [Example 4-6](#) and [Example 4-7](#).



While self joins are supported, you must alias the fields you are interested in to different names beforehand, so they can be accessed.

Example 4-6. Table of pandas and sizes

Name	Size
Happy	1.0

⁵ If the number of distinct keys is too high, you can also use `reduceByKey`, sort on the value, and take the top k.

Name	Size
Sad	0.9
Happy	1.5
Coffee	3.0

Example 4-7. Table of pandas and zip codes

Name	Zip
Happy	94110
Happy	94103
Coffee	10504
Tea	07012

Spark’s supported join types are **inner**, **left_outer** (aliased as “outer”), **right_outer** and **left_semi**. With the exception of “left_semi” these join types all join the two tables, but they behave differently when handling rows that do not have keys in both tables.

The “inner” join is both the default and likely what you think of when you think of joining tables. It requires that the key be present in both tables, or the result is dropped as shown in **Example 4-8** and **inner join table**.

Example 4-8. Simple inner join

```
// Inner join implicit
df1.join(df2, df1("name") === df2("name"))
// Inner join explicit
df1.join(df2, df1("name") === df2("name"), "inner")
```

Table 4-1. Inner join of df1, df2 on name

Name	Size	Name	Zip
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.5	Happy	94110
Happy	1.0	Happy	94103

Name	Size	Name	Zip
Happy	1.5	Happy	94103

Left outer joins will produce a table with all of the keys from the left table, and any rows without matching keys in the right table will have null values in the fields that would be populated by the right table. Right outer joins are the same, but with the requirements reversed.

Example 4-9. Left outer join

```
// Left outer join explicit
df1.join(df2, df1("name") === df2("name"), "left_outer")
```

Table 4-2. Left outer join df1, df2 on name

Name	Size	Name	Zip
Sad	0.9	null	null
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103

Example 4-10. Right outer join

```
// Right outer join explicit
df1.join(df2, df1("name") === df2("name"), "right_outer")
```

Table 4-3. Right outer join df1, df2 on name

Name	Size	Name	Zip
Sad	0.9	null	null
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103

Name	Size	Name	Zip
null	null	Tea	07012

Left semi joins are the only kind of join which only has values from the left table. A left semi join is the same as filtering the left table for only rows with keys present in the right table.

Example 4-11. Left semi join

```
// Left semi join explicit
df1.join(df2, df1("name") === df2("name"), "leftsemi")
```

Table 4-4. Left semi join

Name	Size
Coffee	3.0
Happy	1.0
Happy	1.5

Self Joins

Self joins are supported on DataFrames; but we end up with duplicated column names. So you can access the results you need to alias the DataFrames to different names. Once you've aliased each DataFrame, in the result you can access the individual columns for each DataFrame with `dfName.colName`.

Example 4-12. Self join

```
val joined = df.as("a").join(df.as("b")).where($"a.name" === $"b.name")
```

Broadcast Hash Joins

In SparkSQL you can see the type of join being performed by calling `queryExecution.executedPlan`. As with core Spark, if one of the tables is much smaller than the other you may want a broadcast hash join. You can hint to Spark SQL that a given DF should be broadcast for join by calling `broadcast` on the DataFrame before joining it (e.g. `df1.join(broadcast(df2), "key")`). Spark also automatically uses the `spark.sql.conf.autoBroadcastJoinThreshold` to determine if a table should be broadcast.

Dataset Joins

Joining Datasets is done with `joinWith`, and this behaves similarly to a regular relational join, except the result is a tuple of the different record types as shown in [Example 4-13](#). This is somewhat more awkward to work with after the join, but also does make self joins, as shown in [Example 4-14](#), much easier, as you don't need to alias the columns first.

Example 4-13. Joining two Datasets

```
val result: Dataset[(RawPanda, CoffeeShop)] = pandas.joinWith(coffeeShops,  
  $"zip" === $"zip")
```

Example 4-14. Self join a Dataset

```
val result: Dataset[(RawPanda, RawPanda)] = pandas.joinWith(pandas,  
  $"zip" === $"zip")
```



Using a self join and a `lit(true)`, you can produce the cartesian product of your Dataset, which can be useful but also illustrates how joins (especially self-joins) can easily result in unworkable data sizes.

As with DataFrames you can specify the type of join desired (e.g. `inner`, `left_outer`, `right_outer`, `leftsemi`), changing how records present only in one Dataset are handled. Missing records are represented by null values, so be careful.

Conclusion

Now that you have explored joins, it's time to focus on transformations and the performance considerations associated with them. For those interested in continuing learning more about Spark SQL, we will continue with Spark SQL tuning in [???](#), where we include more details on join-specific configurations like number of partitions and join thresholds.